ExamIQ: A Smart Examination and Performance Analyzer

A PROJECT REPORT

Submitted To
MS. ASHIMA SOOD

Submitted By
KUSUM
UID: 26MCA20248
26MCA-4B

in partial fulfillment for the award of the degree of

MASTERS OF COMPUTER APPLICATIONS

IN

UNIVERSITY INSTITUTE OF COMPUTING

CHANDIGARH UNIVERSITY

October-November, 2026

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Software & Hardware Requirements
5. Implementation
6. Key Functionalities
7. System Analysis & Differentiation
8. Output & Screenshots
9. Conclusion
10. Future Advancements

Abstract

ExamIQ: A Smart Examination and Performance Analyzer is a three-layer application that converts the routine work of recording and analysing examination marks into a single, verifiable digital workflow. The computational core of the project is a C++17 program (cpp/analytics.cpp) that accepts a student identifier and five subject marks and returns the total, the minimum, the maximum, the average, the grade, the pass or fail verdict and the frequency of every digit in the identifier. The engine is written deliberately as an exercise in computing aptitude: the marks are held in a fixed-size array and traversed through pointers, the same aggregates are recomputed through variadic functions as an independent cross-check, the record is encapsulated in a Student class, the classification rules are expressed with plain conditional statements, and the digit count is produced by a ten-element counter array working on character codes. The application around the engine — a Python 3 Tkinter interface, a database adapter with SQLite and Oracle modes, and an Oracle PL/SQL schema with a computed view and a reporting package — supplies storage, orchestration and presentation. By replacing manual calculation with a compiled, deterministic and testable engine, the system removes arithmetic and transcription errors from routine examination work while keeping every derived value auditable.

Introduction

Computing aptitude is usually examined through small problems: reverse a number, count the frequency of its digits, find the largest and the smallest of a set of values, average a list, and decide a grade from a threshold. In a classroom each of these problems is solved once and then discarded. ExamIQ was designed to place exactly these constructs inside a working application, so that every one of them carries a consequence: a wrong pointer offset, a counter that is not re-initialised or an unvalidated argument now produces a wrong mark statement instead of a wrong answer in a laboratory notebook.

The engine therefore behaves like a small service. It receives its input as command-line arguments, checks the argument count before touching any data, performs the computation with no file access and no global state, and prints a single-line JSON object that the calling application decodes. The desktop client turns a selected student into that argument list, runs the compiled binary, and binds the decoded values to the cards of its analysis screen; the same contract is used by the Flask web variant of the interface.

Two properties were treated as non-negotiable during the design. The computation must be **deterministic** — the same identifier and the same five marks must always produce the same verdict — and the program must **fail honestly**: malformed input produces a documented error object and a non-zero exit status, never a partially correct result. Everything else in the report follows from these two rules.

Objectives

The module was developed to satisfy the following objectives:

1. **Apply fundamental constructs to a real problem**
   To use arrays, pointers, functions, variadic argument lists, classes and one-pass loops on data that a department actually maintains, instead of on isolated laboratory exercises.

2. **Implement correct aggregation and classification**
   To compute the total, minimum, maximum and average of five marks from a single traversal of the array, and to apply the grade rule (A from 400, B from 300, C from 200, otherwise Needs Support) together with the subject-wise pass rule, under which any mark below 33 is a FAIL.

3. **Solve a number-manipulation problem with an array of counters**
   To report how often each decimal digit occurs in the student identifier using a ten-element integer array and character arithmetic, so that the frequency report is produced in a single pass over the identifier.

4. **Design a stable inter-program contract**
   To fix the command-line format, emit one JSON object per invocation and document every exit status, so that the Python client and the compiled engine can be changed independently as long as the contract is preserved.

5. **Keep the implementation efficient and verifiable**
   To hold every routine to linear time over its input with constant extra space, and to cross-check the pointer-based total against the variadic total on the screen where both are displayed.

Software & Hardware Requirements

1. **Software Requirements**

   - **Programming Language:** C++17 for the analytics engine (cpp/analytics.cpp), which uses only the standard headers iostream, string, cstdarg and cstdlib.
   - **Compiler and Toolchain:** g++ 16.2 (MSYS2 / MinGW-w64, UCRT64 environment) or any compiler with C++17 support; the released binary is produced by g++ cpp/analytics.cpp -o analytics.exe -std=c++17.
   - **Calling Application:** Python 3.10 or later. tk_app.py launches the engine with subprocess.run(), decodes the reply with json.loads() and renders the values on the analysis screen; app.py exposes the same contract through a Flask web interface.

- **Data Store:** SQLite 3 through the standard-library sqlite3 module for the demonstration dataset (examiq.db); the Oracle script sql/oracle_setup.sql supplies the same columns when the adapter is started with DB_MODE=oracle.
- **Database Option:** Oracle Database 19c or 21c Express Edition with python-oracledb 2.x for the table EXAMIQ_STUDENTS, the computed view EXAMIQ_STUDENT_RESULTS and the reporting package EXAMIQ_REPORTS.
- **Development Environment:** Visual Studio Code with the C/C++ extension, an MSYS2 terminal, and Git for version control.

2. Hardware Requirements

- **Processor:** any 64-bit x86-64 processor at 2.0 GHz or faster; the engine is a single-threaded console program.
- **Memory:** at least 4 GB of RAM for the complete system; the engine itself allocates under 1 MB at run time.
- **Storage:** 500 MB of free disk space for the toolchain, the sources, the compiled binary and the database.
- **Display and Network:** a 1366 × 768 or larger display for the desktop client; no network access and no database server are needed to run or test the engine.

Implementation

The system follows the **Model-View-Controller (MVC)** design pattern, which separates the application into three interconnected components that handle the internal representation of information, the way that information is presented to the user, and the processing of the user's input. Because the numerical work is deliberately kept out of the interface, the controller role is shared between the Python orchestrator and the compiled engine.

- **Model:** The persistence layer is a normalised table of student records. In SQLite the table students stores an auto-increment key, the unique student code, the name and the five subject columns (mathematics, computer_science, physics, english, dbms); in Oracle the same columns are held by EXAMIQ_STUDENTS, the view EXAMIQ_STUDENT_RESULTS derives total, percentage and grade on the server side, and the package EXAMIQ_REPORTS exposes the class-summary and subject-average procedures.
- **View:** The user interface is built with Python and Tkinter (tk_app.py) on a dark theme. The screens are the Dashboard for class statistics, Students for the record list, Marks Entry for

adding and editing marks, Rankings for the merit list, C++ Analysis for the verdict of the engine and Reports for the class summary and the CSV export.

- **Controller:** Two components divide the work. tk_app.py turns the selected record into the argument list of the engine, runs it with subprocess.run(args, timeout=5, check=True), decodes the JSON reply and binds the values to the interface; cpp/analytics.cpp validates its arguments, constructs a Student object, computes the verdict and serialises it. A missing binary, a non-zero exit status and a malformed reply are handled as three distinct failures.

The input contract

The program requires exactly eight arguments: the command word analyze, the student identifier and five integer marks. The count is validated before any value is parsed, so malformed input is rejected at the boundary and never reaches the data:

```
int main(int argc, char* argv[]) {
    if (argc != 3 + SUBJECTS || string(argv[1]) != "analyze") {
        cout << "{\"error\":\"Usage: analytics analyze STUDENT_ID m1 m2 m3 m4 m5\"}";
        return 1;                                  // rejected before any data is touched
    }
    string studentId = argv[2];
    int marks[SUBJECTS];
    for (int i = 0; i < SUBJECTS; i++) marks[i] = atoi(argv[3 + i]);

    Student s(studentId, marks);                   // object created from validated input
    int digitCount[10];
    s.digitFrequency(digitCount);                  // ten counters, one pass
```

Arrays and pointers

The five subject marks live in a fixed-size array inside the class. Because the name of an array decays to a pointer to its first element, the aggregation functions accept const int* and walk the marks with pointer arithmetic, which keeps the traversal explicit and makes the bounds the only thing that has to be checked:

```
// arr points to marks[0]; *(arr + i) is the i-th element
int totalUsingPointer(const int* arr, int n) {
    int total = 0;
    for (int i = 0; i < n; i++) total += *(arr + i);
    return total;
}

double averageUsingPointer(const int* arr, int n) {
    return (double)totalUsingPointer(arr, n) / n;      // cast before division
}
```

Variadic functions

A second, independent route to the same aggregates uses variable argument lists from cstdarg. The first argument tells the function how many values follow, and va_start, va_arg and va_end

walk the list. Keeping both implementations active is a practical correctness check — the two totals must always agree, and the analysis screen displays them side by side:

```
int sumOf(int count, ...) {
    va_list args;
    va_start(args, count);
    int total = 0;
    for (int i = 0; i < count; i++) total += va_arg(args, int);
    va_end(args);
    return total;
}
// minOf() and maxOf() follow the same pattern: the first value seeds
// the accumulator, the remaining values are compared one by one.
```

Class, conditions and the digit-frequency algorithm

The Student class encapsulates the identifier and the marks array and exposes const member functions for every derived value, so a caller cannot modify a record while reading its statistics. The verdict rules are plain conditional statements, and the frequency count uses the classic digit-array technique: the character code of a digit is shifted by the code of '0' to obtain an index between 0 and 9.

```
class Student {
private:
    string id;
    int marks[SUBJECTS];
public:
    int total() const         { return totalUsingPointer(marks, SUBJECTS); }
    int variadicTotal() const {
        return sumOf(5, marks[0], marks[1], marks[2], marks[3], marks[4]);
    }

    string grade() const {                        // threshold classification
        int t = total();
        if (t >= 400) return "A";
        else if (t >= 300) return "B";
        else if (t >= 200) return "C";
        return "Needs Support";
    }
    string result() const {                       // any subject below 33 fails
        for (int i = 0; i < SUBJECTS; i++)
            if (marks[i] < 33) return "FAIL";
        return "PASS";
    }
    void digitFrequency(int digitCount[10]) const {
        for (int d = 0; d < 10; d++) digitCount[d] = 0;         // initialise
        for (size_t i = 0; i < id.length(); i++)
            if (id[i] >= '0' && id[i] <= '9')
                digitCount[id[i] - '0']++;                      // character arithmetic
    }
};
```

The output contract

The verdict is printed as a single JSON object, and only non-zero digit counts are emitted, which keeps the payload small and makes it readable in a terminal. The exit status is 0 on success and 1 on any usage error:

```
> analytics.exe analyze EQ1001 88 91 84 79 95
{"total":437,"variadic_total":437,"minimum":79,"maximum":95,"average":87.4,
 "grade":"A","result":"PASS","digit_frequency":{"0":2,"1":2}}
> echo %ERRORLEVEL%
0
```

For a real class roll number the digit counters are equally direct: the identifier 26MCA2002 reports 0 → 2, 2 → 3 and 6 → 1, which is exactly the frequency of its decimal digits and is computed without converting the identifier into a number.

Cost of the computation

Let n be the number of subjects (five in this project) and k the number of characters in the identifier. Every routine performs one pass over its input, so the whole engine is linear in n + k and uses constant extra space apart from the two fixed arrays:

Routine	Technique	Time
total / average	pointer traversal of the marks array	O(n)
sumOf / minOf / maxOf	variadic walk with va_arg	O(n)
grade	chained comparison against fixed thresholds	O(1)
result	early-exit loop over the marks	O(n)
digitFrequency	ten counters and character arithmetic	O(k)
class ranking (Python)	sorted() with a key over m records	O(m log m)

The only non-linear step in the complete system is the ranking of the class, which sorts the student records and therefore costs O(m log m); it belongs to the Python layer because the compiled engine works on one student at a time. This division keeps the engine stateless and trivially testable, while the interpreted layer handles the collection-level operations where sorting and dictionaries are more convenient.

Key Functionalities

The system is organised into two modules.

1. **Engine Module (computation)**

    - **Command-line interface:** analytics.exe analyze STUDENT_ID m1 m2 m3 m4 m5, with the argument count validated before anything is parsed.

- **Aggregate statistics:** total, minimum, maximum and average computed from one traversal of the marks array and recomputed through variadic functions as a cross-check.
- **Classification:** the grade rule on the total and the subject-wise pass rule, both expressed as explicit conditional logic.
- **Digit-frequency analysis:** a ten-element counter array reports how often each digit occurs in the student identifier, in one pass and without numeric conversion.
- **Machine-readable output:** a single JSON object on standard output, with exit status 0 on success and 1 on any usage error, and deterministic behaviour because the engine keeps no state and reads no files.

2. Application Module (orchestration and presentation)

- **Guarded process invocation:** the client builds the argument list, runs the engine with a five-second timeout and check=True, and converts every failure mode into a readable message.
- **JSON decoding:** json.loads() turns the engine output into a dictionary that the interface binds directly to its cards, so no string parsing is duplicated in the interface.
- **Class-level aggregation:** totals, percentages, grades, ranks, the pass rate and the five subject averages are computed in Python from the stored rows; the distribution is bucketed into five bands with int(pct // 20).
- **Visual verification:** the analysis screen shows the pointer-based total and the variadic total next to each other, so any inconsistency between the two implementations is visible immediately.
- **Report export:** the Reports screen summarises the class and exports a CSV file with rank, student code, name, subject marks, total, percentage and grade; in Oracle mode the same figures can be produced by EXAMIQ_REPORTS.

System Analysis & Differentiation

ExamIQ stands out from spreadsheet-based mark registers and from online practice platforms because its numerical core is written rather than configured:

- **Compiled computation instead of scripting:** the numerical core is a native binary with no interpreter start-up cost and no runtime dependency beyond the standard library, which makes the difference between compiled and interpreted execution measurable during a demonstration.

- **An explicit contract instead of shared memory:** the two languages communicate only through the documented command line and one JSON object, so either side can be rewritten independently as long as the contract is preserved.
- **Deliberate redundancy as verification:** the pointer traversal and the variadic aggregation compute the same total by different mechanisms, and the interface displays both; this is a cheap and effective way to catch an off-by-one error or an accumulation defect.
- **Failure-aware design:** a missing binary, a non-zero exit status and malformed JSON are three distinct, handled cases rather than three crashes.
- **Fixed-size arrays with bounded input:** the marks array and the ten digit counters are sized at compile time, so the engine performs no dynamic allocation and cannot suffer a memory-management error.
- **Clear separation of concerns:** the engine answers one question about one student, and the Python layer answers questions about the class; neither layer duplicates the responsibility of the other.

Output & Screenshots

The screenshots below were captured from the running application. The analysis screen shows the verdict computed by the compiled engine for the first record of the sample dataset.

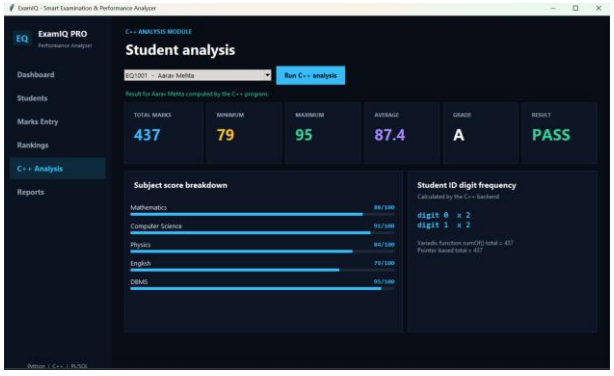


Figure 1: C++ Analysis — verdict computed by the compiled engine



Figure 2: Dashboard — class statistics and score distribution

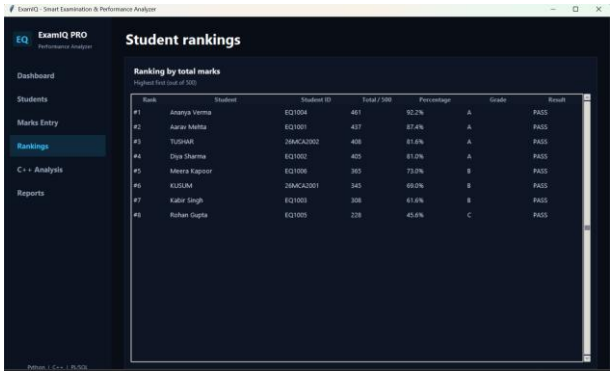


Figure 3: Rankings — merit list by total marks

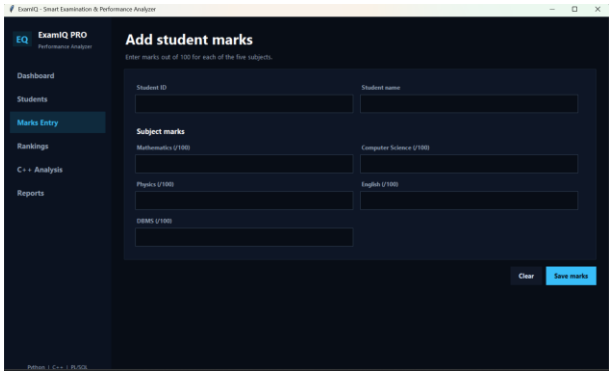


Figure 4: Marks Entry — add-student form for five subjects

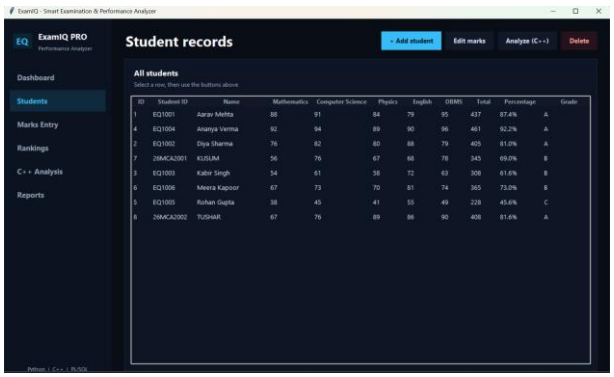


Figure 5: Students — stored records with derived totals



Figure 6: Reports — class summary and CSV export

Github Upload

The complete source of the project — the C++ engine, the build command, the Python client and the Oracle script — is published in the following repository:

Github Link

https://github.com/kusumbharadwaj22-ai/ExamIQ

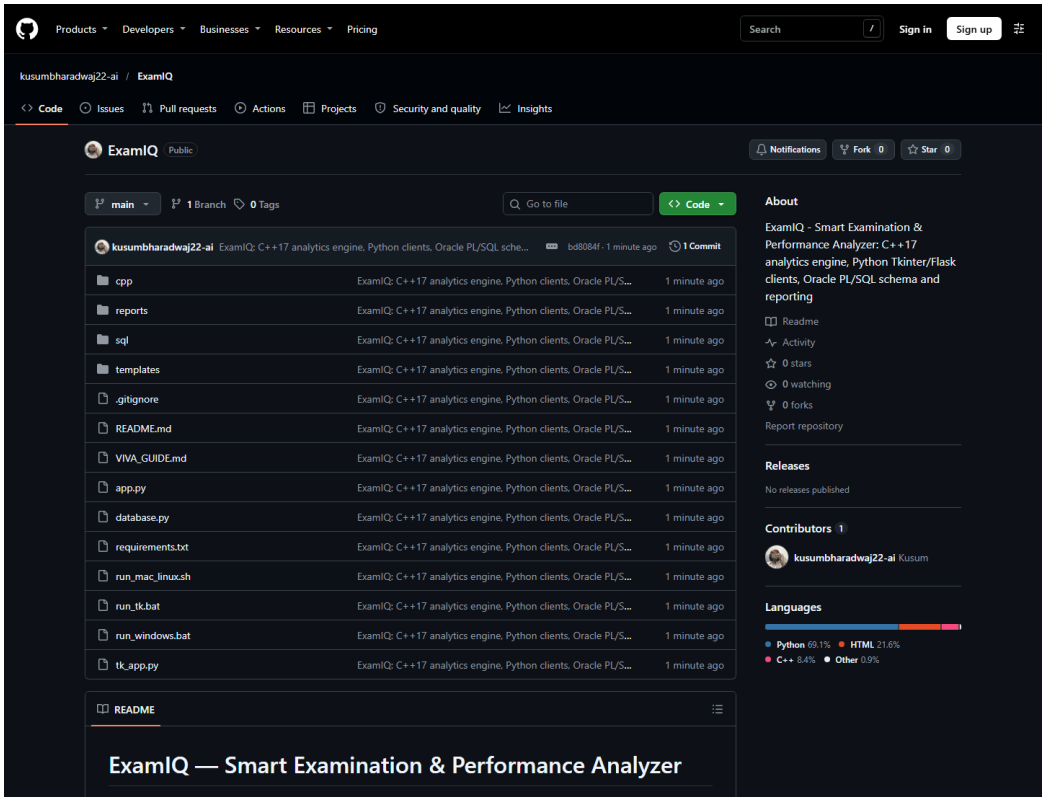


Figure 7: GitHub repository overview — source, project report and version control history

Conclusion

The ExamIQ analytics engine shows that a modest program can exercise the full range of computing aptitude: arrays and pointers for storage and traversal, variadic functions for flexible argument lists, a class for encapsulation, conditions for classification, character arithmetic for a number-manipulation problem, and a strict input and output contract for inter-program communication. Every routine performs a single pass in linear time with constant extra space, and the two independent implementations of the total act as a built-in correctness check. Around the engine, the Tkinter client, the database adapter and the Oracle objects turn the same computation into a system a department can use: marks are stored once, statistics are derived rather than typed, and the merit list, the class summary and the CSV export always agree with the records.

The project also changed the character of the exercise. Instead of printing a verdict for a human reader, the program had to produce output that another program could consume reliably; instead of assuming well-formed input, it had to validate its arguments and report failure through its exit status. That shift — from a program that works when everything goes right to a component that behaves predictably when something goes wrong — is the most transferable outcome of the work.

Future Advancements

To keep ExamIQ competitive and academically useful, the following upgrades are planned:

- **Single source of truth for the rules:** move the grade thresholds and the pass rule into one configuration shared by the C++ engine, the database adapter and the view, so that a change of policy is made once.
- **Batch mode with standard input:** accept a complete class through stdin or a file argument, so that one invocation computes every student instead of one process per student.
- **Dynamic subject count and richer statistics:** replace the compile-time SUBJECTS constant with a count argument and dynamic allocation, and add standard deviation, median and a subject-difficulty index derived from z-scores.
- **Automated unit tests:** run the engine over fixed inputs and compare the JSON output with expected values, so that the contract is verified on every commit.
- **Portable build and reporting:** provide a CMake configuration and a continuous-integration job for Windows, Linux and macOS, and extend the export beyond CSV to a formatted PDF mark statement per student.